

Bài tập thực hành tuần 8

Họ và tên: Nguyễn Văn Phúc Huy

MSSV: 23110163

Lớp: 23TTH (Chiều thứ 6, ca 1)

Cài đặt thuật toán Mahalanobis k-means

```
class MahalanobisKMeans:
```

```
    def __init__(self, n_clusters=3, max_iter=100, tol=1e-4, reg_covar=1e-6):
```

```
        self.n_clusters = n_clusters
```

```
        self.max_iter = max_iter
```

```
        self.tol = tol
```

```
        self.reg_covar = reg_covar
```

```
    def fit(self, X):
```

```
        n_samples, n_features = X.shape
```

```
        # 1. Khởi tạo ngẫu nhiên tâm cụm (centroids)
```

```
        random_idx = np.random.permutation(n_samples)[:self.n_clusters]
```

```
        self.centroids = X[random_idx]
```

```
        # Khởi tạo ma trận nghịch đảo hiệp phương sai (ma trận đơn vị) và log_det = 0
```

```
        self.inv_covariances = [np.eye(n_features) for _ in range(self.n_clusters)]
```

```
        self.log_dets = np.zeros(self.n_clusters)
```

```
        self.labels = np.zeros(n_samples)
```

```
        for iteration in range(self.max_iter):
```

```
            old_centroids = self.centroids.copy()
```

```
            # 2. Bước Gán (Assignment): Tính khoảng cách Mahalanobis + Log_det penalty
```

```
            distances = np.zeros((n_samples, self.n_clusters))
```

```
            for k in range(self.n_clusters):
```

```
                diff = X - self.centroids[k]
```

```
                # Mahalanobis distance + ln(|Sigma|)
```

```
                mahalnobis_dist = np.sum(np.dot(diff, self.inv_covariances[k]) * diff, axis=1)
```

```
                distances[:, k] = mahalnobis_dist + self.log_dets[k]
```

```
            self.labels = np.argmin(distances, axis=1)
```

```
            # 3. Bước Cập nhật (Update)
```

```
            for k in range(self.n_clusters):
```

```
                cluster_data = X[self.labels == k]
```

```
                if len(cluster_data) > 1:
```

```
                    self.centroids[k] = np.mean(cluster_data, axis=0)
```

```
                    cov_matrix = np.cov(cluster_data, rowvar=False)
```

```
                    cov_matrix += np.eye(n_features) * self.reg_covar
```

```
                    self.inv_covariances[k] = np.linalg.inv(cov_matrix)
```

```
                    # Tính Logarit định thức một cách an toàn (tránh underflow/overflow)
```

```
                    sign, log_det = np.linalg.slogdet(cov_matrix)
```

```
                    self.log_dets[k] = log_det
```

```
            # 4. Kiểm tra hội tụ
```

```
            shift = np.linalg.norm(self.centroids - old_centroids)
```

```
            if shift < self.tol:
```

```
                break
```

```
        return self
```

```
    def predict(self, X):
```

```
        n_samples = X.shape[0]
```

```
        distances = np.zeros((n_samples, self.n_clusters))
```

```
        for k in range(self.n_clusters):
```

```
            diff = X - self.centroids[k]
```

```

mahalanobis_dist = np.sum(np.dot(diff, self.inv_covariances[k]) * diff, axis=1)
# Khi predict dữ liệu mới cũng phải cộng log_det của cụm đó
distances[:, k] = mahalanobis_dist + self.log_dets[k]

```

```

return np.argmin(distances, axis=1)

```

Class `MahalanobisKMeans` đã được định nghĩa để thực thi thuật toán k-means sử dụng khoảng cách Mahalanobis. Hoạt động như sau:

- Hàm `__init__` khởi tạo đối tượng với số lượng cụm, số lần lặp tối đa, ngưỡng hội tụ và hệ số điều chỉnh cho ma trận hiệp phương sai.

```

def __init__(self, n_clusters=3, max_iter=100, tol=1e-4, reg_covar=1e-6):
    self.n_clusters = n_clusters
    self.max_iter = max_iter
    self.tol = tol
    self.reg_covar = reg_covar

```

- Hàm `fit` thực hiện quá trình huấn luyện mô hình trên dữ liệu đầu vào X. Hoạt động bằng cách:
 - Khởi tạo ngẫu nhiên các tâm cụm (centroids) từ dữ liệu.
 - Khởi tạo ma trận nghịch đảo hiệp phương sai và log_det cho mỗi cụm.
 - Trong mỗi vòng lặp, thực hiện hai bước chính:
 - Bước Gán (Assignment): Tính khoảng cách Mahalanobis cộng với log_det penalty cho mỗi điểm dữ liệu đến mỗi cụm và gán nhãn cho điểm dữ liệu dựa trên cụm gần nhất.
 - Bước Cập nhật (Update): Cập nhật tâm cụm bằng cách tính trung bình của các điểm dữ liệu thuộc cụm đó, tính ma trận hiệp phương sai và nghịch đảo của nó, và cập nhật log_det.
 - Lặp qua các bước gán và cập nhật cho đến khi hội tụ hoặc đạt số lần lặp tối đa.

```

def fit(self, X):
    n_samples, n_features = X.shape

    # 1. Khởi tạo ngẫu nhiên tâm cụm (centroids)
    random_idx = np.random.permutation(n_samples)[:self.n_clusters]
    self.centroids = X[random_idx]

    # Khởi tạo ma trận nghịch đảo hiệp phương sai (ma trận đơn vị) và Log_det = 0
    self.inv_covariances = [np.eye(n_features) for _ in range(self.n_clusters)]
    self.log_dets = np.zeros(self.n_clusters)
    self.labels = np.zeros(n_samples)

    for iteration in range(self.max_iter):
        old_centroids = self.centroids.copy()

        # Bước Gán (Assignment): Tính khoảng cách Mahalanobis + Log_det penalty
        distances = np.zeros((n_samples, self.n_clusters))
        for k in range(self.n_clusters):
            diff = X - self.centroids[k]
            # Mahalanobis distance + ln(|Sigma|)
            mahalanobis_dist = np.sum(np.dot(diff, self.inv_covariances[k]) * diff, axis=1)
            distances[:, k] = mahalanobis_dist + self.log_dets[k]

        self.labels = np.argmin(distances, axis=1)

        # Bước Cập nhật (Update)
        for k in range(self.n_clusters):
            cluster_data = X[self.labels == k]

            if len(cluster_data) > 1:
                self.centroids[k] = np.mean(cluster_data, axis=0)
                cov_matrix = np.cov(cluster_data, rowvar=False)
                cov_matrix += np.eye(n_features) * self.reg_covar

                self.inv_covariances[k] = np.linalg.inv(cov_matrix)

            # Tính Logarit định thức (tránh underflow/overflow)
            sign, log_det = np.linalg.slogdet(cov_matrix)
            self.log_dets[k] = log_det

        # Kiểm tra hội tụ
        shift = np.linalg.norm(self.centroids - old_centroids)
        if shift < self.tol:

```

`break`

`return self`

- Hàm `predict` dự đoán nhãn cụm cho dữ liệu mới bằng cách tính khoảng cách Mahalanobis cộng với log_det penalty và trả về nhãn của cụm gần nhất.

```
def predict(self, X):
    n_samples = X.shape[0]
    distances = np.zeros((n_samples, self.n_clusters))
    for k in range(self.n_clusters):
        diff = X - self.centroids[k]
        mahalanobis_dist = np.sum(np.dot(diff, self.inv_covariances[k]) * diff, axis=1)
        # Khi predict dữ liệu mới cũng phải cộng log_det của cụm đó
        distances[:, k] = mahalanobis_dist + self.log_dets[k]

    return np.argmin(distances, axis=1)
```

Nhận xét gì về GMM, thuật toán k-means và Mahalanobis k-means

Về thuật toán thì

- K-means: Thuật toán này đơn giản, dễ hiểu và dễ triển khai. Tuy nhiên, nó có nhược điểm là chỉ tìm được các cụm hình cầu và không thể xử lý tốt các cụm có hình dạng phức tạp hoặc có kích thước khác nhau. K-means cũng nhạy cảm với các điểm ngoại lai (outliers) và có thể bị ảnh hưởng bởi chúng.
- GMM (Gaussian Mixture Model): Thuật toán này linh hoạt hơn K-means vì nó cho phép các cụm có hình dạng khác nhau và có thể xử lý tốt các cụm có kích thước khác nhau. GMM sử dụng phân phối Gaussian để mô hình hóa dữ liệu, do đó nó có thể tìm được các cụm có hình dạng phức tạp. Tuy nhiên, GMM cũng nhạy cảm với các điểm ngoại lai và có thể bị ảnh hưởng bởi chúng.
- Mahalanobis k-means: Thuật toán này cải tiến từ K-means bằng cách sử dụng khoảng cách Mahalanobis thay vì khoảng cách Euclid. Khoảng cách Mahalanobis tính đến sự phân bố của dữ liệu và các mối quan hệ giữa các biến, do đó nó có thể xử lý tốt các cụm có hình dạng khác nhau và kích thước khác nhau. Tuy nhiên, thuật toán này cũng nhạy cảm với các điểm ngoại lai và có thể bị ảnh hưởng bởi chúng.

Về đồ thị thì

- K-means với GMM cho kết quả gần như tương tự nhau. Nhưng GMM phân bố dữ liệu tốt hơn K-means, vì nó cho phép các cụm có hình dạng khác nhau và có thể xử lý tốt các cụm có kích thước khác nhau. Trong khi đó, K-means chỉ tìm được các cụm hình cầu và không thể xử lý tốt các cụm có hình dạng phức tạp hoặc có kích thước khác nhau.
- Mahalanobis k-means cho kết quả khác biệt hơn so với K-means và GMM. Thuật toán này sử dụng khoảng cách Mahalanobis, do đó nó có hình dạng elip rõ rệt hơn so với K-means và GMM. Điều này cho thấy Mahalanobis k-means có khả năng xử lý tốt các cụm có hình dạng khác nhau và kích thước khác nhau, trong khi K-means và GMM chỉ tìm được các cụm hình cầu.